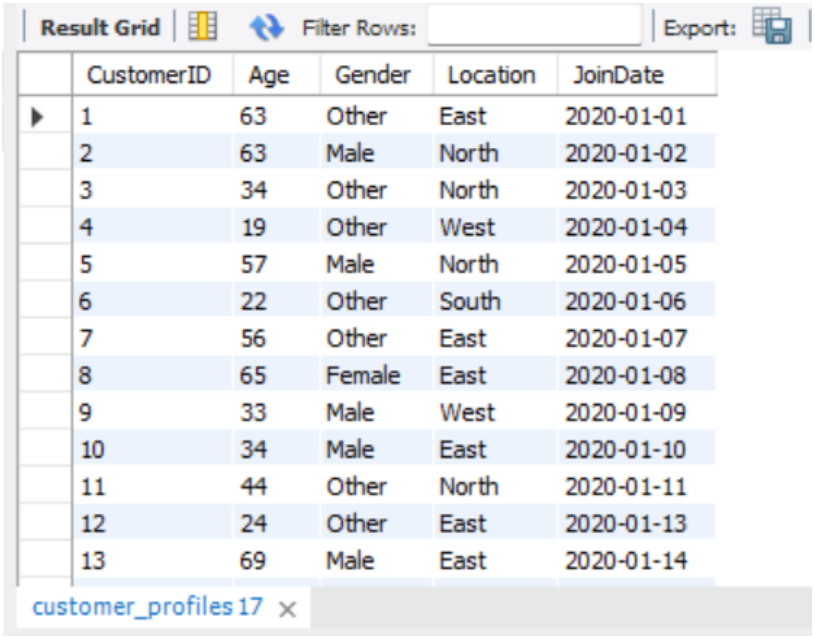
SCREENSHOTS OF

SQL QUERRY OUTPUTS

1. Select Statement - Used to retrieve specific columns or rows from a table

select * from customer_profiles;

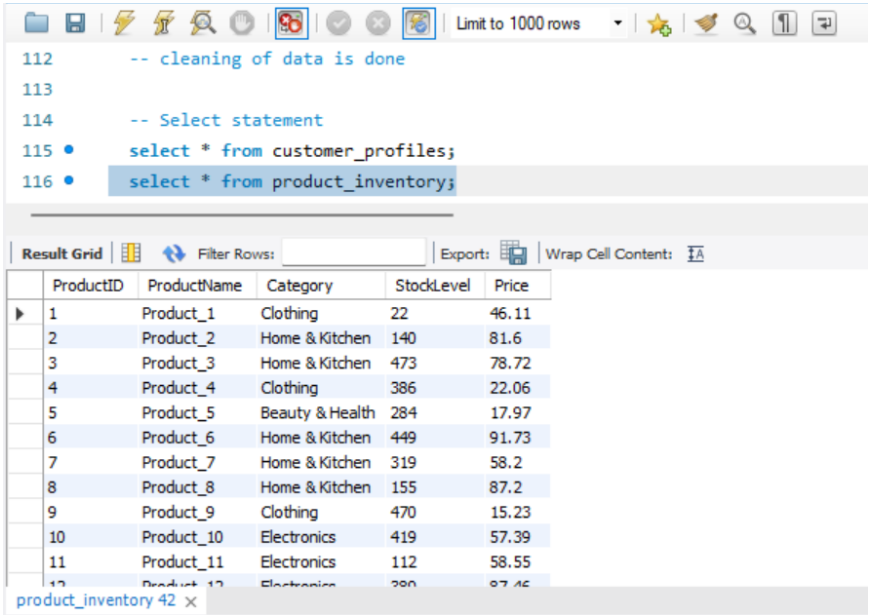


Result Grid | Filter Rows: | Export:

CustomerID	Age	Gender	Location	JoinDate
1	63	Other	East	2020-01-01
2	63	Male	North	2020-01-02
3	34	Other	North	2020-01-03
4	19	Other	West	2020-01-04
5	57	Male	North	2020-01-05
6	22	Other	South	2020-01-06
7	56	Other	East	2020-01-07
8	65	Female	East	2020-01-08
9	33	Male	West	2020-01-09
10	34	Male	East	2020-01-10
11	44	Other	North	2020-01-11
12	24	Other	East	2020-01-13
13	69	Male	East	2020-01-14

customer_profiles 17

select * from product_inventory;



Limit to 1000 rows

```
112    -- cleaning of data is done
113
114    -- Select statement
115    select * from customer_profiles;
116    select * from product_inventory;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	Category	StockLevel	Price
1	Product_1	Clothing	22	46.11
2	Product_2	Home & Kitchen	140	81.6
3	Product_3	Home & Kitchen	473	78.72
4	Product_4	Clothing	386	22.06
5	Product_5	Beauty & Health	284	17.97
6	Product_6	Home & Kitchen	449	91.73
7	Product_7	Home & Kitchen	319	58.2
8	Product_8	Home & Kitchen	155	87.2
9	Product_9	Clothing	470	15.23
10	Product_10	Electronics	419	57.39
11	Product_11	Electronics	112	58.55

product_inventory 42

select * from sales_transaction;

```
116    -- Select statement
117    select * from customer_profiles;
118    select * from product_inventory;
119    select * from sales_transaction;
120
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows

TransactionID	CustomerID	ProductID	QuantityPurchased	TransactionDate	Price
1	103	120	3	2023-01-01	30.43
2	436	126	1	2023-01-01	15.19
3	861	55	3	2023-01-01	67.76
4	271	27	2	2023-01-01	65.77
5	107	118	1	2023-01-01	14.55
6	72	53	1	2023-01-01	26.27
7	701	39	2	2023-01-01	95.92
8	21	65	4	2023-01-01	17.19
9	615	145	4	2023-01-01	66
10	122	158	2	2023-01-01	22.27
11	467	181	2	2023-01-01	69

sales_transaction 61

2. Where - Filters rows based on a given condition

-- Q. we have to show all customers from 'west' location

```
120
121    -- Use Of WHERE and ORDER BY
122    -- let's say we have to show all customers from 'west' location
123    select *
124    from customer_profiles
125    where Location = 'west';
126
```

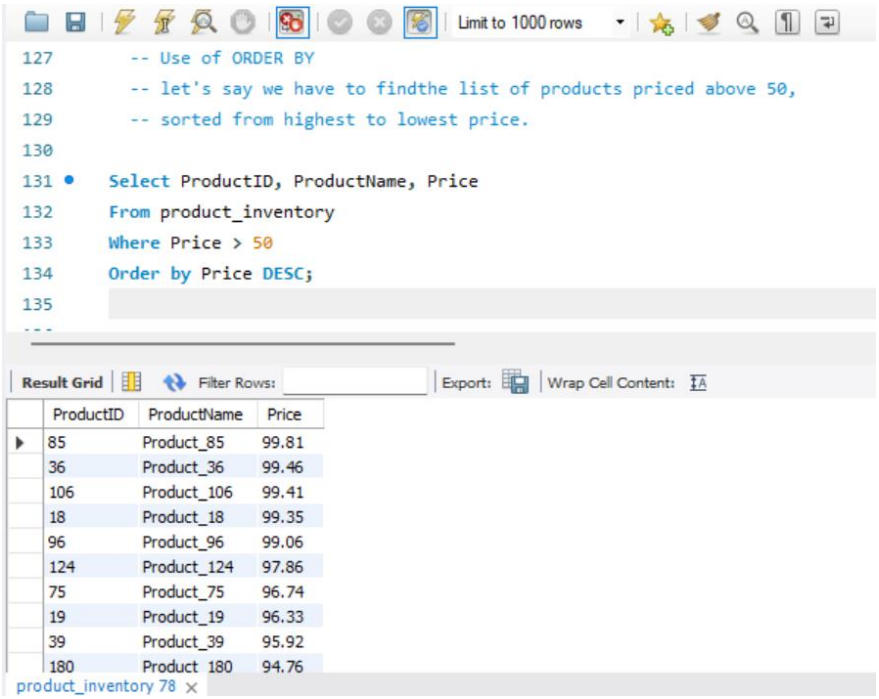
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	Age	Gender	Location	JoinDate
4	19	Other	West	2020-01-04
9	33	Male	West	2020-01-09
21	55	Other	West	2020-01-22
23	32	Other	West	2020-01-25
25	55	Female	West	2020-01-27
29	26	Male	West	2020-01-31
31	63	Female	West	2020-02-02
32	36	Female	West	2020-02-04
37	48	Other	West	2020-02-09

customer_profiles 75

3. ORDER BY - Sorts the result set in ascending or descending order.

-- Q. find the list of products priced above 50, sorted from highest to lowest price.



```
127     -- Use of ORDER BY
128     -- let's say we have to findthe list of products priced above 50,
129     -- sorted from highest to lowest price.
130
131 •   Select ProductID, ProductName, Price
132     From product_inventory
133     Where Price > 50
134     Order by Price DESC;
135
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

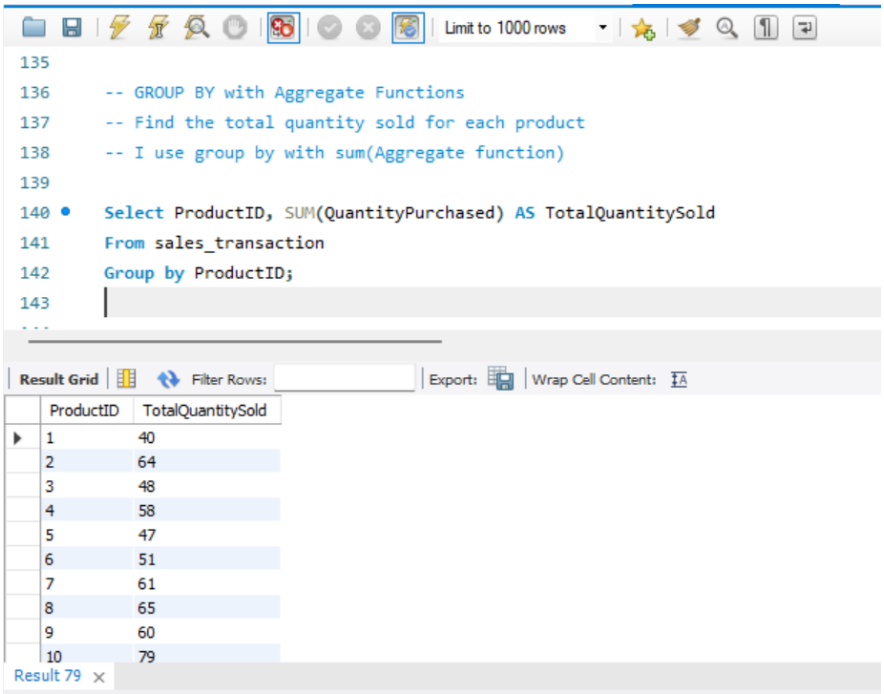
ProductID	ProductName	Price
85	Product_85	99.81
36	Product_36	99.46
106	Product_106	99.41
18	Product_18	99.35
96	Product_96	99.06
124	Product_124	97.86
75	Product_75	96.74
19	Product_19	96.33
39	Product_39	95.92
180	Product_180	94.76

product_inventory 78

4. GROUP BY with Aggregate Functions - Groups rows with the same values for aggregate calculations

- SUM

-- Q. Find the total quantity sold for each product.



```
135
136     -- GROUP BY with Aggregate Functions
137     -- Find the total quantity sold for each product
138     -- I use group by with sum(Aggregate function)
139
140 •   Select ProductID, SUM(QuantityPurchased) AS TotalQuantitySold
141     From sales_transaction
142     Group by ProductID;
143
```

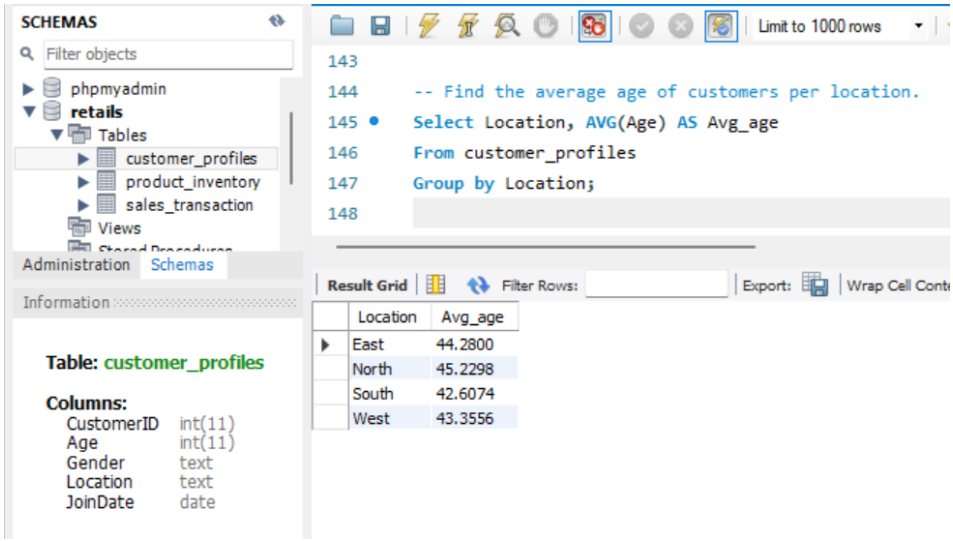
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	TotalQuantitySold
1	40
2	64
3	48
4	58
5	47
6	51
7	61
8	65
9	60
10	79

Result 79

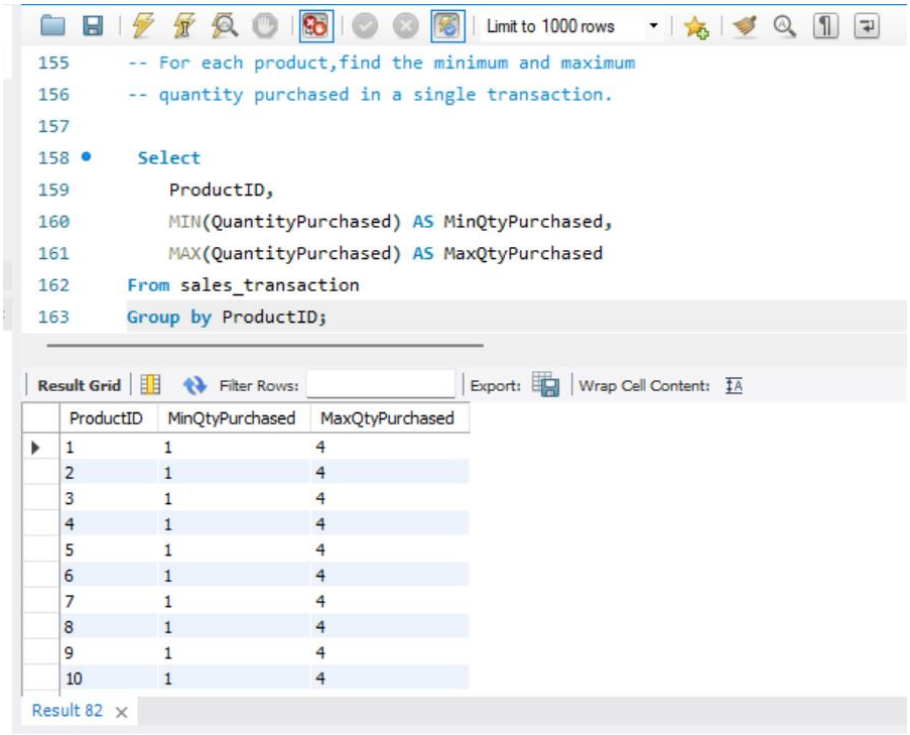
- AVG

-- Q. Find the average age of customers per location.



- MIN & MAX

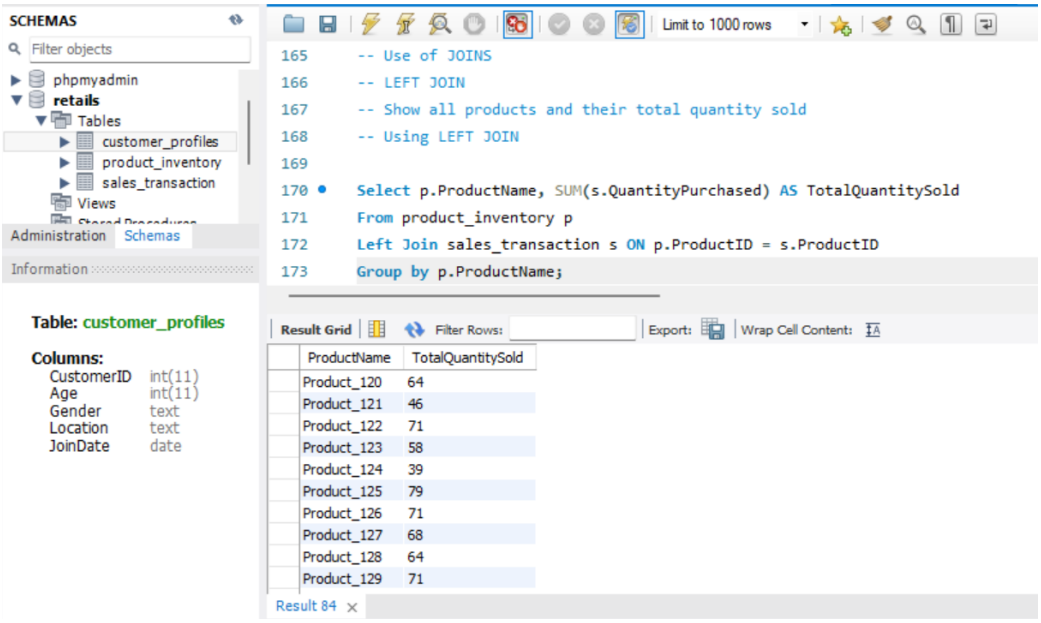
-- Q. For each product, find the minimum and maximum quantity purchased in a single transaction.



5. JOINS - JOIN is a clause that combines rows from two or more tables based on a related column between them.

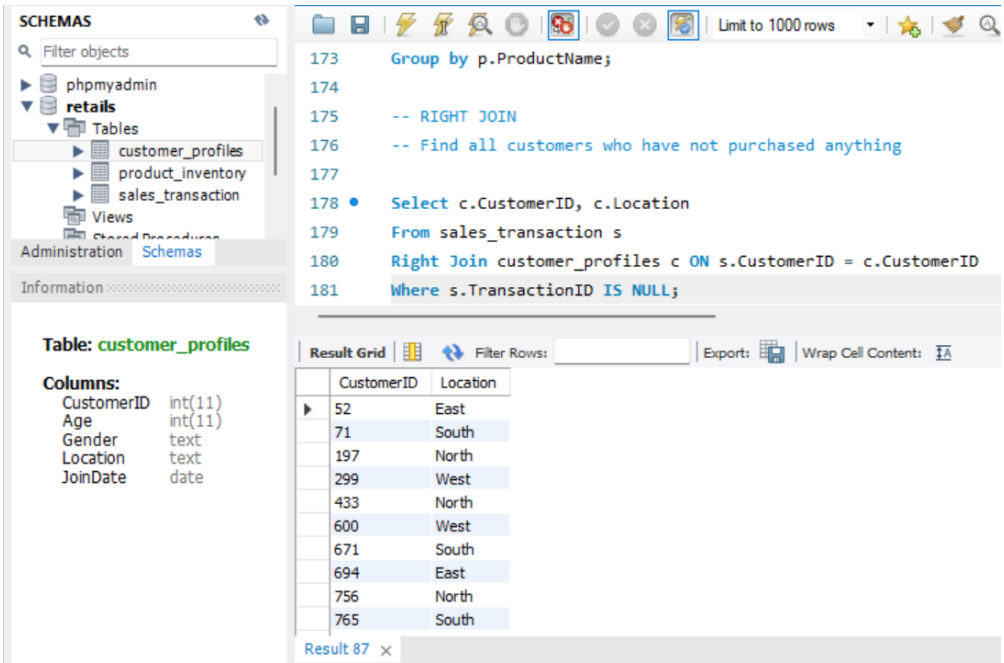
- LEFT JOIN - Returns all rows from the left table & matching rows from the right table (NULL if no match).

-- Q. Show all products and their total quantity sold



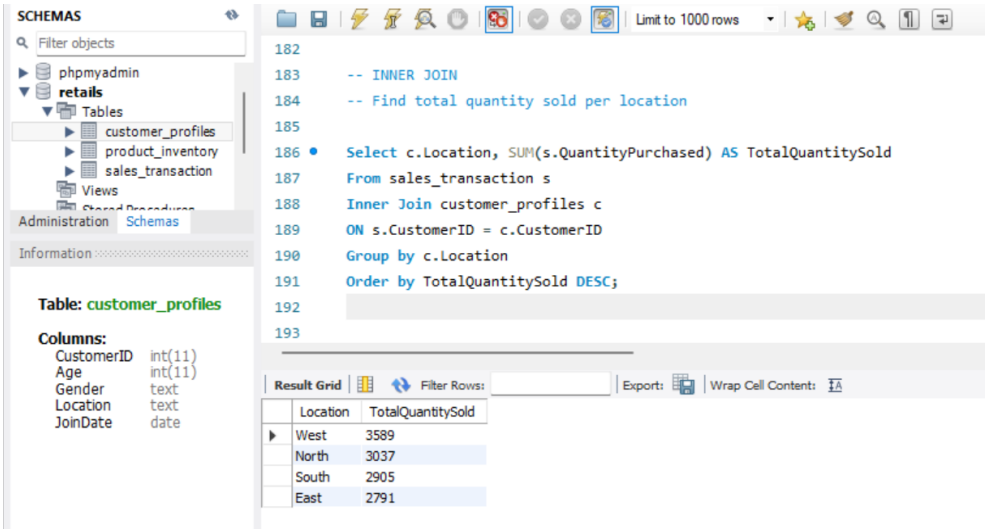
- RIGHT JOIN - Returns all rows from the right table & matching rows from the left table (NULL if no match).

-- Q. Find all customers who have not purchased anything



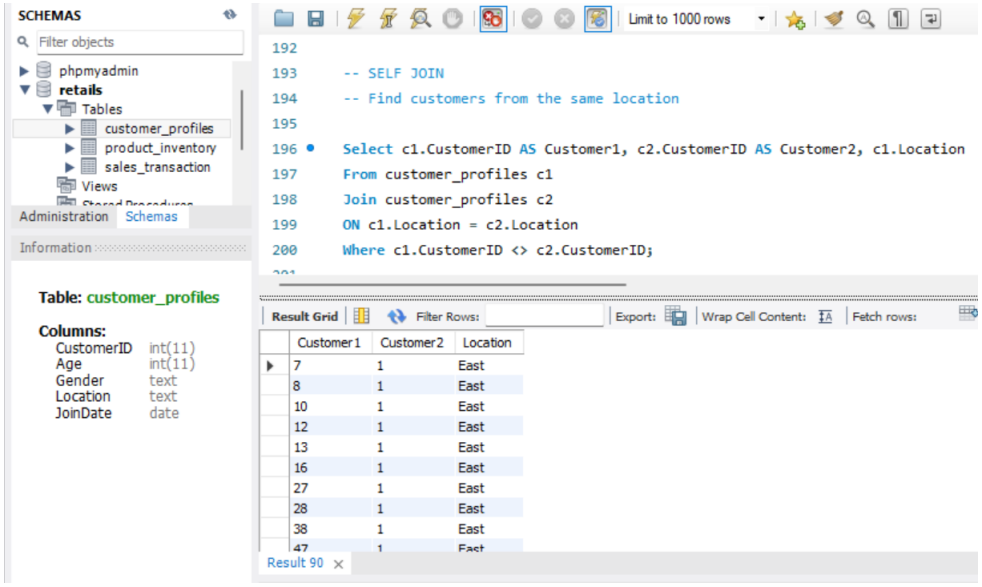
- INNER JOIN - Returns rows where there is a match in both tables.

-- Q. Find total quantity sold per location



- SELF JOIN - Joins a table to itself to compare rows within the same table.

-- Q. Find customers from the same location



6. SUBQUERIES - A query nested inside another query to provide intermediate results

-- Q. Get all customers who spent more than the average spending of all customers

```sql
-- USE OF SUBQUERIES
-- Get all customers who spent more than the average spending of all customers

Select CustomerID,ROUND(SUM(Price * QuantityPurchased),2) AS TotalSpent,
    ROUND((Select AVG(CustomerTotal)
        From (Select SUM(Price * QuantityPurchased) AS CustomerTotal
            From sales_transaction
            Group by CustomerID) AS avg_spending), 2) AS AverageSpending
From sales_transaction
Group by CustomerID
Having TotalSpent > (
    Select AVG(CustomerTotal)
    From (
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	TotalSpent	AverageSpending
1	1188.04	709.81
2	895.57	709.81
7	1383.86	709.81
8	1297.25	709.81
12	1426.31	709.81
13	927.24	709.81
15	748.95	709.81

Result 3

-- Q. Find the top 3 most expensive products using a subquery in Where.

```sql
Select ProductName, Price
From product_inventory
Where Price IN (
    Select Price
    From (
        Select Price
        From product_inventory
        Order by Price DESC
        Limit 3
    ) AS top_prices
)
Order by Price DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	Price
Product_85	99.81
Product_36	99.46
Product_106	99.41

7. VIEWS - A virtual table created from a query, used to simplify complex queries and reuse logic.

-- Q. Create a view to show customer spending summary & their result

```
242    -- Creating Views
243    -- views - A virtual table created from a query,used to simplify complex queries and reuse logic.
244    -- Create a view to show customer spending summary
245    Create View customer_spending_summary AS
246    Select c.CustomerID, c.Location,
247           SUM(s.Price * s.QuantityPurchased) AS TotalSpent,
248           COUNT(s.TransactionID) AS TotalTransactions
249    From customer_profiles c
250    Left Join sales_transaction s ON c.CustomerID = s.CustomerID
251    Group by c.CustomerID, c.Location;
252
253    -- querying the created views
254    Select * From customer_spending_summary
255    Order by TotalSpent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	Location	TotalSpent	TotalTransactions
936	East	2834.4700000000003	12
664	South	2519.04	14
670	East	2432.15	12
39	West	2221.29	12
435	North	2158.98	10

customer_spending_summary 15

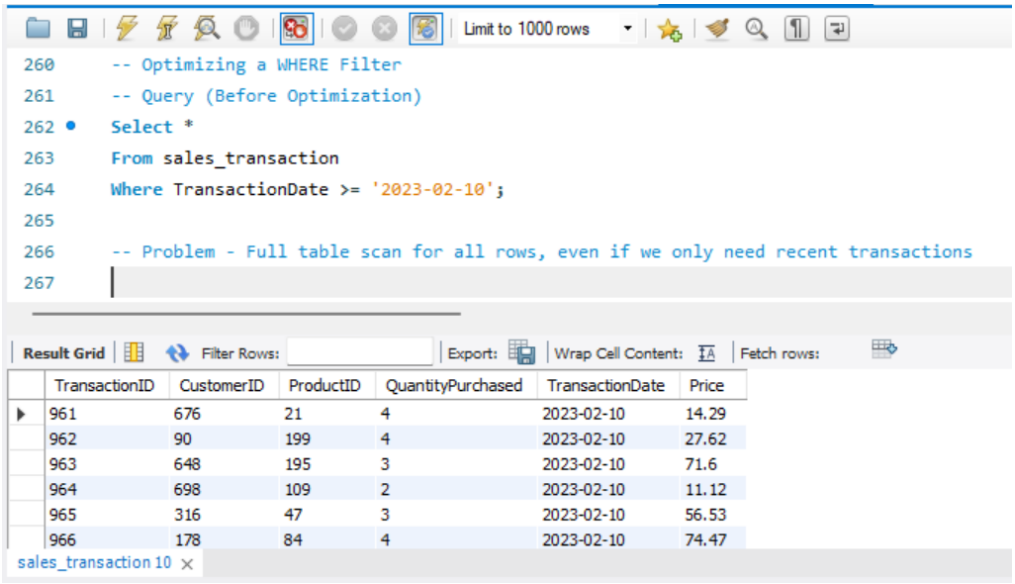
8. INDEXES - Indexing in SQL involves creating a data structure to improve the speed of data retrieval operations from a database table.

-- Create indexes to speed up joins and filtering.

```
251    -- Creating indexes for optimization
252    -- indexes
253
254    -- Create indexes to speed up joins and filtering.
255
256    Create Index idx_customer_id ON sales_transaction(CustomerID);
257    Create Index idx_product_id ON sales_transaction(ProductID);
258    Create Index idx_location ON customer_profiles(Location);
```

-- Optimizing a WHERE Filter Using Indexing

-- Query (Before Optimization)



```
260    -- Optimizing a WHERE Filter
261    -- Query (Before Optimization)
262 •  Select *
263    From sales_transaction
264    Where TransactionDate >= '2023-02-10';
265
266    -- Problem - Full table scan for all rows, even if we only need recent transactions
267
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

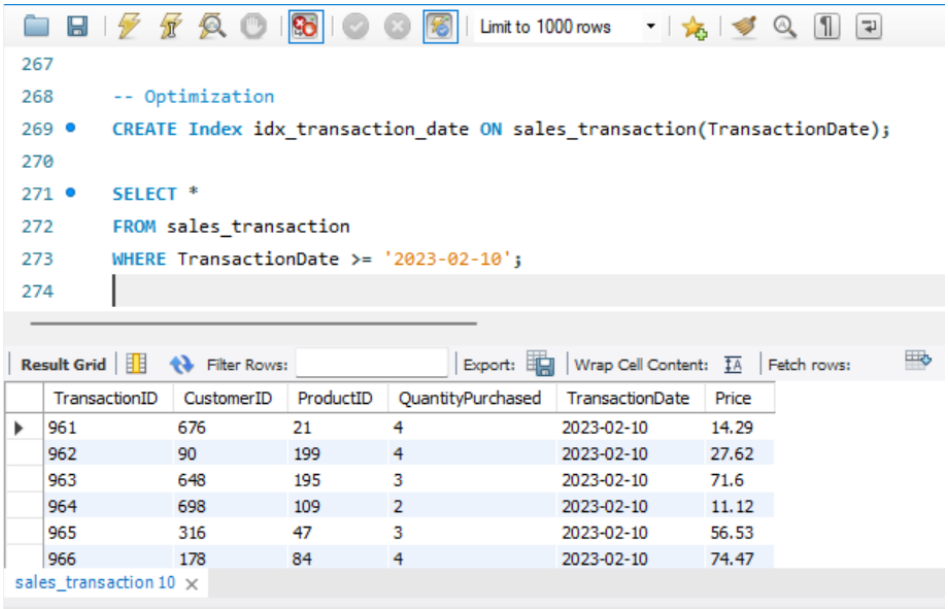
TransactionID	CustomerID	ProductID	QuantityPurchased	TransactionDate	Price
961	676	21	4	2023-02-10	14.29
962	90	199	4	2023-02-10	27.62
963	648	195	3	2023-02-10	71.6
964	698	109	2	2023-02-10	11.12
965	316	47	3	2023-02-10	56.53
966	178	84	4	2023-02-10	74.47

sales_transaction 10

-- Problem – Full table scan for all rows, even if we only need recent transactions date

-- The query take 0.015 sec time duration to process & 0.016 sec time in Fetch the result

-- After Optimization



```
267
268    -- Optimization
269 •  CREATE Index idx_transaction_date ON sales_transaction(TransactionDate);
270
271 •  SELECT *
272    FROM sales_transaction
273    WHERE TransactionDate >= '2023-02-10';
274
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

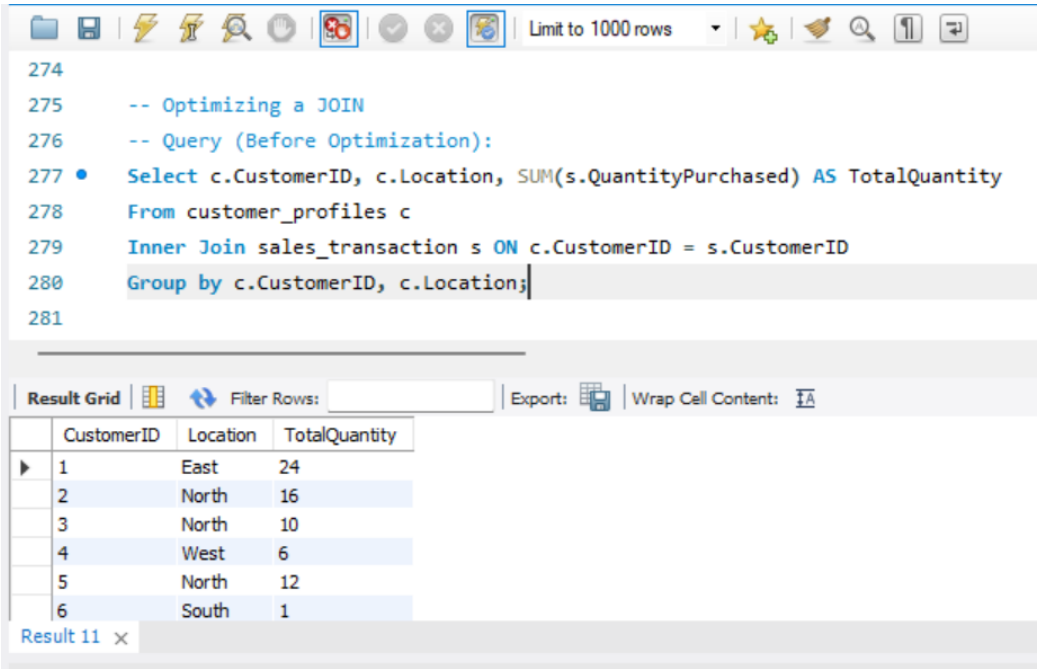
TransactionID	CustomerID	ProductID	QuantityPurchased	TransactionDate	Price
961	676	21	4	2023-02-10	14.29
962	90	199	4	2023-02-10	27.62
963	648	195	3	2023-02-10	71.6
964	698	109	2	2023-02-10	11.12
965	316	47	3	2023-02-10	56.53
966	178	84	4	2023-02-10	74.47

sales_transaction 10

-- Now the query take 0.000 sec time duration to process & 0.016 sec time in Fetch the result

-- Optimizing a JOIN Using Indexing

-- Query (Before Optimization)



Problem - Joins are slow if sales_transaction is big.

-- **After Optimization**